

STL Containers — Խորը ուղեցույց

C++ հարցազրույցի ամբողջական STL presentation (vector, list, և ավելին)

Այս ուղեցույցը C++ STL container-ների խորը վերլուծություն է՝ հասկապես **vector vs list** (ներքին կառուցվածք, memory layout, complexity, cache locality, iterator invalidation), բոլոր կարևոր մեթոդներով, plus մյուս container-ների (deque, map/set, unordered) համեմատություն: Վերջում՝ interview հարցերի ամփոփ:

0. STL Container-ների ընդհանուր պատկեր

```
// Sequence containers
vector    // dynamic array
list      // doubly linked list
deque     // double-ended queue
array     // fixed-size array (C++11)
forward_list // singly linked list

// Associative (sorted, tree-based)
set / multiset
map / multimap

// Unordered (hash-based, C++11)
unordered_set / unordered_multiset
unordered_map / unordered_multimap

// Container adaptors
stack     // LIFO
queue     // FIFO
priority_queue // heap
```

Հիմնական թեման՝ **vector vs list**, դա interview-ի ամենահաճախ հարցն է:

1. Vector vs List — ներքին կառուցվածք

Vector = Dynamic Array (contiguous memory)

```
vector<int> v = {10, 20, 30, 40};
```

Բոլոր տարրերը կողք-կողքի են հիշողությունում.

```
+-----+-----+-----+-----+
| 10 | 20 | 30 | 40 |
+-----+-----+-----+-----+
անընդմեջ (contiguous)
```

Vector-ը ներսում պահում է 3 pointer/արժեք. `begin`, `end` (size), `capacity`.

List = Doubly Linked List (scattered memory)

```
list<int> lst = {10, 20, 30, 40};
```

Ամեն node ունի `data`, `next`, `prev`.

```
Node(10) <-> Node(20) <-> Node(30) <-> Node(40)
```

Node-ները հիշողությունում ցրված են (scattered), ոչ պարտադիր կողք-կողքի.

```
Node(10) --next--> Node(20) --next--> Node(30)
```

2. Size vs Capacity & Reallocation

Interview-ի ամենակարևոր հարցերից.

```
vector<int> v;  
v.push_back(10);  
v.push_back(20);  
v.push_back(30);  
// size = 3, capacity = 4 (հնարավոր)
```

- `v.size()` — իրական տարրերի քանակն է.
- `v.capacity()` — հատկացված memory-ի չափն է (սովորաբար ավելի մեծ):

Reallocation (թանկ գործողություն)

Երբ capacity-ն լցվում է և ավելացնում ես.

```
capacity = 4:  [1][2][3][4]  
push_back(5):  տեղ չկա ->  
1. կոռ (մեծ) memory վերցնում  
2. capacity մեծացնում (սովորաբար x2)  
3. բոլոր տարրերը copy/move  
4. հին memory ազատում  
Result:       [1][2][3][4][5]
```

Reallocation-ը $O(n)$ է (բոլորն պետք copy/move լինեն): Amortized-ով push_back-ը $O(1)$ է, քանի որ x2 աճը հազվադեպ է լինում:

reserve() — reallocation-ից խուսափելու

```
vector<int> v;  
v.reserve(1000); // նախապես 1000-ի memory վերցրու  
for (int i = 0; i < 1000; i++)  
    v.push_back(i); // ոչ մի reallocation
```

Եթե նախապես գիտես չափը, `reserve()` -ը զգալի արագացնում է:

3. Complexity համեմատություն

Գործողություն	vector	list
Random access <code>v[i]</code>	O(1)	O(n) (index չկա)
<code>push_back</code>	O(1) amortized	O(1)
<code>push_front</code>	O(n)	O(1)
<code>insert middle</code>	O(n)	O(1)*
<code>erase middle</code>	O(n)	O(1)*
<code>search (unsorted)</code>	O(n)	O(n)

* list-ի insert/erase-ը **O(1)** է միայն եթե iterator-ն արդեն ունես. այդ դիպքին հասնելը (`advance`) ինքնին **O(n)** է:

Vector access` O(1)

```
cout << v[500];    // ուղիղ հասցեական հաշվարկ` base + 500*sizeof(int)
```

List access` O(n)

```
auto it = lst.begin();
advance(it, 5);    // 5 քայլ պետք է անել, index չկա
cout << *it;
```

4. Cache Locality (interview favorite)

Սա բացատրում է, թե ինչու vector-ը գործնականում հաճախ ավելի արագ է, նույնիսկ երբ complexity-ն նույնն է թվում.

Vector — տարրերը կողք-կողքի են.

```
[1][2][3][4][5]    -> CPU cache-ը մեկ բեռնում շատ տարբեր է կարդում  
                    -> քիչ cache miss -> արագ
```

List — node-ները ցրված են.

```
[1] ... [2] ..... [3] .. [4]    -> ամեն node առանձին memory տեղ  
                                   -> շատ cache miss -> դանդաղ
```

Դրա համար. **modern hardware-ի վրա vector-ը հաճախ հաղթում է list-ին** նույնիսկ insert-ի դեպքում, քանի որ contiguous memory-ն շատ ավելի cache-friendly է:

5. Iterator Invalidation

Հաճախ հարցնում են, թե երբ են iterator-ները «կոտրվում».

Vector — reallocation-ի ժամանակ invalid

```
vector<int> v = {1, 2, 3};
auto it = v.begin();
v.push_back(4);           // եթե reallocation եղավ ->
cout << *it;             // it-ը ԱՅԼԵՎՍ ՎԱՎԵՐ ՉԷ (dangling)
```

Vector-ի insert/erase/push_back-ը (reallocation-ի դեպքում) invalidate է անում iterator-ները.

List — stable iterators

```
list<int> lst = {1, 2, 3};
auto it = lst.begin();
lst.push_back(4);        // it-ը ՄՆՈՒՄ Է ՎԱՎԵՐ
cout << *it;             // OK
```

List-ում insert/push-ը չի invalidate անում ուրիշ iterator-ները (միայն erase-ը՝ ջոյնված node-ինը): Սա list-ի մեծ առավելություններից մեկն է:

6. Memory Usage

Vector

Ամեն `int` ~ 4 bytes (plus capacity-ի ավելցուկ memory, բայց մեկ բլոկ):

```
[4][4][4][4]    -> ~4 bytes/element
```

List

Ամեն node = data + 2 pointer (64-bit).

```
data: 4 bytes + next: 8 bytes + prev: 8 bytes = ~24 bytes/element (+alignment)
```

List-ը 5-6 անգամ ավելի memory է ունում ամեն `int` -ի համար:

7. Vector-ի կարևոր մեթոդներ

Ստեղծում

```
vector<int> v;           // դառնալ
vector<int> v(5);        // 5 հատ 0
vector<int> v(5, 10);    // 10 10 10 10 10
vector<int> v = {1,2,3,4}; // initializer list
```

Element access

```
v[0];           // O(1), bounds check ՉԿԱ
v.at(0);        // O(1), bounds check ԿԱ (throw out_of_range)
v.front();      // առաջին
v.back();       // վերջին
```

Add / Remove

```
v.push_back(10);    // վերջում ավելացնում
v.emplace_back(10);  // in-place կառուցում (efficient object-ների համար)
v.pop_back();        // վերջինի ջոխում
v.insert(v.begin()+2, 100); // O(n)
v.erase(v.begin()+2); // O(n)
v.clear();           // բոլորի ջոխում (size=0)
```

`push_back` vs `emplace_back`. `push_back`-ը object է ստեղծում ապա copy/move անում;
`emplace_back`-ը ուղիղ container-ի ներսում է կառուցում argument-ներից (խնայում copy/move).

```
vector<string> names;
names.push_back("Arman"); // temporary string + move
names.emplace_back("Arman"); // ուղիղ կառուցում, ավելի efficient
```

Size functions

```
v.size();           // տարրերի քանակ
v.empty();          // դառնա՞րե՞ք
v.capacity();        // հաճախված memory
v.reserve(1000);     // նախապես memory
v.shrink_to_fit();   // ավելցուկ capacity ազատում
```

Iterators & loops

```
for (auto it = v.begin(); it != v.end(); ++it)
    cout << *it << " ";

for (auto x : v)           // range-based (copy)
    cout << x << " ";

for (auto& x : v)          // reference (փոփոխվելու համար ՊԱՐՏԱԴԻՐ)
    x *= 2;
```

Algorithms (`#include <algorithm>`)

```
sort(v.begin(), v.end());           // աճման
sort(v.begin(), v.end(), greater<int>()); // նվազման
reverse(v.begin(), v.end());

auto it = find(v.begin(), v.end(), 20);
if (it != v.end()) cout << "Found";

binary_search(v.begin(), v.end(), 10); // sorted լինելու պայմանով
count(v.begin(), v.end(), 5);
*min_element(v.begin(), v.end());
*max_element(v.begin(), v.end());
```

Erase-Remove idiom (ՇԱՏ սիրված interview հարց)

`remove` -ը իրականում չի ջոխում միայն շարժում է «ջոխվել» տարրերը վերջ և վերադարձնում նոր `logical end`; `erase` -ն է իրական ջոխում.

```
v.erase(remove(v.begin(), v.end(), 10), v.end()); // բոլոր 10-երը ջոխում
```

Unique (sorted-ից հետո)

```
sort(v.begin(), v.end());  
v.erase(unique(v.begin(), v.end()), v.end()); // կրկնապատկերները ջնջում
```

8. List-ի կարևոր մեթոդներ

```
list<int> lst;
lst.push_back(10);    // վերջում
lst.push_front(5);    // սկզբում (O(1)՝ vector-ում O(n))
lst.pop_back();
lst.pop_front();
lst.front();
lst.back();
lst.remove(10);        // բոլոր 10-երը ջնջում
lst.sort();            // list-ի սեփական sort (std::sort չի օգտագործում՝ random access չկա)
lst.reverse();
lst.unique();          // ծանր կոմպլեքս ջնջում
```

splice — list-ի սուպեր ուժ

Մի list-ից մյուս տեղափոխում է առանց արժեքի կոպիի, O(1)։

```
list<int> a = {1,2,3};
list<int> b = {4,5,6};
a.splice(a.end(), b);    // b-ն ամբողջությամբ տեղափոխվում a-ի վերջ, O(1)
// a = {1,2,3,4,5,6}, b դատարկ
```

Կարևոր. `list.sort()` օգտագործում է, ոչ `std::sort`-ը, քանի որ `std::sort`-ը random access iterator է պահանջում, որ list-ը չունի։

9. Մյսս container-ների արագ ամփոփ

deque (double-ended queue)

```
deque<int> d;  
d.push_back(1);    // O(1)  
d.push_front(0);   // O(1)  <- vector-ից տարբեր, արագ երկու ծայրում  
d[5];              // O(1) random access
```

deque = vector-ի random access + երկու ծայրում արագ insert. Բայց memory-ն ոչ լրիվ contiguous է (block-երի):

map / set (tree, sorted)

```
map<string, int> m;  
m["age"] = 25;    // key sorted, O(log n)  
set<int> s;  
s.insert(5);      // O(log n), ավտոմատ sorted, unique
```

Red-Black Tree ներքում. բոլոր գործողությունները $O(\log n)$, iteration` sorted կարգով.

unordered_map / unordered_set (hash)

```
unordered_map<string, int> um;  
um["age"] = 25;    // O(1) average, O(n) worst (hash collision)  
unordered_set<int> us;  
us.insert(5);      // O(1) average
```

Hash table ներքում. $O(1)$ average, բայց ոչ sorted:

map vs unordered_map

```
map          -> Red-Black Tree, sorted,    O(log n),    կարգի պետք է  
unordered_map -> Hash Table,      unsorted, O(1) avg,   արագությամբ պետք է
```

Container adaptors

```
stack<int> st;    st.push(1); st.top(); st.pop();    // LIFO
queue<int> q;     q.push(1); q.front(); q.pop();    // FIFO
priority_queue<int> pq;  pq.push(3); pq.top();    // max-heap
```

10. Երբ որոնք օգտագործել

vector	-> DEFAULT choice; random access, iteration, sorting, performance
list	-> չատ հաշիւ insert/delete մեղքտեղում + iterator stability լիւրք է
deque	-> երկու ծայրում push/pop (queue-ի լիւրք)
array	-> fixed size, stack-ի վրա, zero overhead
map/set	-> sorted կարգ լիւրք է, $O(\log n)$
unordered_map	-> ամենաարագ lookup, կարգ լիւրք չէ, $O(1)$ avg
stack/queue	-> LIFO / FIFO
priority_queue	-> միշտ ամենաբարձր/կրկն լիւրք է (heap)

Կանոն. **90% դեպքում vector-ն է ճիշտ ընտրությունը**. սկսիր vector-ից, վերջին միայն երբ չափանշանը անհրաժեշտ է, որ ուրիշն ավելի լավ է:

11. Interview հարցեր & պատասխաններ

vector vs list — հիմնական տարբերություն?

vector = dynamic array, contiguous memory, $O(1)$ random access, $O(n)$ middle insert, cache-friendly, memory-efficient. list = doubly linked list, scattered memory, $O(n)$ access, $O(1)$ insert/erase (iterator-ով), iterator stability, ավելի փոքր memory.

size vs capacity?

size = իրական տարրերի քանակ; capacity = հասանելի memory. capacity $\geq$ size միշտ.

reserve() ինչի համար?

*Նախապես memory հատկապես **reallocation-ից** խուսափելու; զիտեմալս **verjnakn chapy** zgali aracamum.*

push_back vs emplace_back?

*push_back **object** ստեղծում ապա **copy/move**; emplace_back **unix container-um** karucum argument-ic (ավելի efficient).*

Vector reallocation ինչ է?

capacity levelis՝ nor mec memory, capacity x2, bolor tarrery copy/move, hin azatum. $O(n)$, amortized push_back $O(1)$.

Ինչու vector հաճախ ավելի արագ է քան list?

Cache locality. contiguous memory -> qich cache miss; list-i scattered node-nery -> shat cache miss.

Iterator invalidation?

*vector **reallocation-ի** dam bolor iterator-ner invalid; list insert/push chi invalidate (miayn erase-y jnjvac node-iny).*

Erase-remove idiom?

`v.erase(remove(v.begin(), v.end(), x), v.end())` — remove-y sharzum e jnjveluner verj, erase-y iskakan jnjum.

map vs unordered_map?

`map` *RB-tree, sorted, $O(\log n)$* ; `unordered_map` *hash, unsorted, $O(1)$ average*.

list һәм linked list ы?

Doubly linked list (forward_list-y singly).

Уфенакаревor sentence-y interview-in:

In modern C++, `std::vector` is usually preferred over `std::list` because of better cache locality, lower memory overhead, and generally better real-world performance.

Cheat Sheet

VECTOR (dynamic array, contiguous):

access $O(1)$ | push_back $O(1)^*$ | insert/erase mid $O(n)$ | cache-friendly | DEFAULT

LIST (doubly linked, scattered):

access $O(n)$ | insert/erase $O(1)$ w/ iterator | iterator-stable | more memory | splice $O(1)$

SIZE vs CAPACITY: size = elements | capacity = allocated ($\geq$ size)

RESERVE: pre-allocate -> avoid reallocation

REALLOCATION: full -> new mem, x2 cap, copy/move all, free old ($O(n)$)

CACHE LOCALITY: vector contiguous -> fast; list scattered -> cache miss

ITERATOR INVALID: vector realloc -> all invalid; list -> stable (except erased)

push_back vs emplace_back: emplace = in-place construct (efficient)

erase-remove: v.erase(remove(begin, end, x), end())

CONTAINERS:

deque -> $O(1)$ both ends | map/set -> RB-tree sorted $O(\log n)$

unordered_* -> hash $O(1)$ avg | stack LIFO | queue FIFO | priority_queue heap

RULE: default to vector; change only when profiling justifies it